

COLLEGE OF CONTROL SCIENCE AND
ENGINEERING

控制科学与工程学院

PROJECT REPORT

黑白棋智能体程序报告

基于位棋盘、迭代加深 PVS 与启发式评估的 Mini AlphaGo 实现

姓 名: 王鹏远

学 号: 3230105936

课程名称: 人工智能与机器学习

项目题目: Mini AlphaGo: 黑白棋

主要方法: Bitboard / PVS / Alpha-Beta

2026 年 4 月 22 日

目录

1	问题重述	1
2	设计思想	2
2.1	总体路线	2
2.2	位棋盘建模	2
2.3	搜索框架	3
2.4	评估函数与动作排序	4
2.5	动态时间与残局精算	5
3	代码内容	6
4	实验结果	7
5	总结	9
A	完整提交源码	10

1 问题重述

本实验任务是为标准 8×8 黑白棋 (Reversi/Othello) 实现一个自动落子程序。平台在每一步调用 `AIPlayer.get_move(board)`，程序需要读取当前棋盘，在全部合法位置中选择一步并返回如 E6 这样的坐标；如果当前玩家无合法着法，则返回 `None`。棋盘使用 X 表示黑棋、O 表示白棋，黑白双方轮流落子，合法落子必须在至少一个方向夹住对方棋子并完成翻转。

黑白棋的难点在于前中期的局部吃子数并不等同于最终优势。角点、边线、行动力、危险的 X 位和 C 位、前沿子数量都会影响后续局面。因此，本项目的目标不是简单选择当前翻子最多的一步，而是在有限时间内尽量评估每一步对后续局势的影响，使智能体在初级、中级和高级机器人对战中均能稳定获胜。

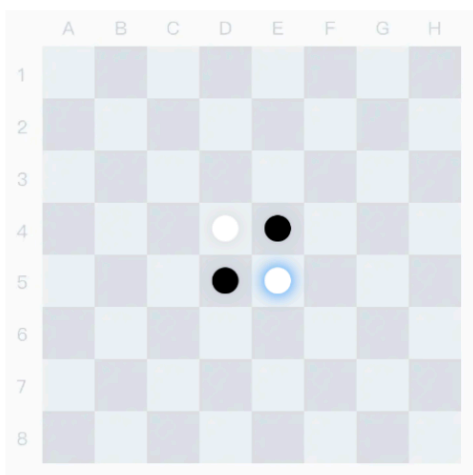


图 1.1: 黑白棋初始局面示意

2 设计思想

2.1 总体路线

课程材料中给出的基本思路是使用蒙特卡洛树搜索，通过选择、扩展、模拟和回传逐步估计各个动作的价值。在实际实现过程中，我先保留了这种“向前看若干步再评价局面”的思想，但将随机模拟替换为确定性的博弈树搜索：当前程序采用两个 64 位整数表示棋盘，并在此基础上进行迭代加深 Principal Variation Search (PVS)。它可以看作是在 Alpha-Beta 剪枝上加入主变优先和零窗口试探，从而在相同时间内搜索更深的分支。

最终策略由四个部分组成：

- **位棋盘表示**：用整数位运算完成合法步生成、翻转和邻接格计算，减少棋盘复制开销；
- **迭代加深 PVS**：从浅层到深层逐步搜索，超时回退到上一层完整结果；
- **置换表与动作排序**：缓存已搜索局面，并优先搜索角点、置换表推荐步和高位置权重步；
- **分阶段启发式评估**：开局重视行动力，中盘重视角和危险位，残局增加棋子差与奇偶性权重。

2.2 位棋盘建模

程序将棋盘从二维列表转换为两个整数：一个保存当前方棋子，一个保存对手棋子。第 r 行第 c 列对应第 $8r + c$ 个二进制位。八个搜索方向被统一写成 DIRECTIONS 表，配合边界掩码避免跨行移位。这样，合法步生成和落子翻转都可以复用同一个方向移动函数。

Code Listing 1: 八方向位移和合法步生成

```
1 DIRECTIONS = (  
2     (1, KEEP_FROM_A), (-1, KEEP_FROM_H),  
3     (8, FULL_BOARD), (-8, FULL_BOARD),  
4     (9, KEEP_FROM_A), (7, KEEP_FROM_H),  
5     (-7, KEEP_FROM_A), (-9, KEEP_FROM_H),  
6 )  
7  
8 def __shift(bits, step, file_mask):  
9     if step > 0:  
10         return ((bits << step) & file_mask) & FULL_BOARD
```

```

11     return (bits >> -step) & file_mask
12
13 def __legal_moves(us, them):
14     blank = FULL_BOARD ^ (us | them)
15     choices = 0
16     for step, file_mask in DIRECTIONS:
17         captured_line = __shift(us, step, file_mask) & them
18         for __ in range(5):
19             captured_line |= __shift(captured_line, step, file_mask) & them
20             choices |= __shift(captured_line, step, file_mask) & blank
21     return choices & FULL_BOARD

```

由于黑白棋每个方向最多连续经过 6 个对方棋子，合法步生成中展开固定次数即可覆盖所有情况。与直接遍历二维数组相比，位棋盘写法把大量判断压缩为按位与、或、移位，适合在搜索树中反复调用。

2.3 搜索框架

主流程首先处理无子可下、唯一合法步和可占角点这三类简单情况。若局面仍需搜索，则根据剩余空位数分配时间，清空置换表，从深度 1 开始迭代加深。每一层搜索完成后更新当前最优步；若本层尚未完成便超时，程序捕获 `SearchTimeout` 并返回上一层结果，从而保证平台调用始终有合法输出。

Code Listing 2: 主决策流程伪代码

```

1 def get_move(board):
2     us, them = board_to_bitboards(board)
3     legal = legal_moves(us, them)
4     if legal is empty:
5         return None
6     if legal has only one move:
7         return that move
8     if legal contains a corner:
9         return best corner move
10
11     deadline = now + time_budget(empty_count)
12     best = first move after move ordering
13     for depth from 1 to depth_limit:
14         candidate = root_search(us, them, depth, best)
15         if this depth finishes:
16             best = candidate
17     return best

```

非根节点采用 Negamax 形式的 PVS。每个节点先查置换表；若缓存深度足够，则根据精确值、下界或上界直接返回或收紧搜索窗口。对第一个候选步使用完整窗口搜索，对后续步先用零窗口判断是否可能超过当前 α ，只有可能成为新主变时才重新以完整窗口搜索。

Code Listing 3: PVS 搜索核心

```

1 for i, move in enumerate(ordered):
2     next_us, next_them = __play(us, them, move)
3     if i == 0:
4         value = -self.__pvs(next_them, next_us, depth - 1,
5                             -beta, -alpha, False)
6     else:
7         value = -self.__pvs(next_them, next_us, depth - 1,
8                             -alpha - 1, -alpha, False)
9         if alpha < value < beta:
10            value = -self.__pvs(next_them, next_us, depth - 1,
11                                -beta, -value, False)
12    alpha = max(alpha, value)
13    if alpha >= beta:
14        break

```

2.4 评估函数与动作排序

评估函数并不单纯比较棋子数，而是综合七类特征：行动力、角点、角邻压力、位置权重、前沿子、边线、棋子差以及残局奇偶性。不同阶段的权重如下表所示。

表 2.1: 分阶段评估权重

特征	开局 > 40 空位	中盘 17-40 空位	残局 ≤ 16 空位
行动力	10	8	5
角点	20	30	35
角邻压力	12	10	4
位置权重	4	3	2
前沿子	3	3	4
边线	1	2	4
棋子差	0	2	12
奇偶性	0	0	8

动作排序与评估函数相互配合。程序优先搜索角点和置换表推荐步，对 X 位和 C

位施加强惩罚，同时估计落子后对手合法步数量。排序质量越高，Alpha-Beta 剪枝越早发生，PVS 的零窗口试探也越容易命中。

Code Listing 4: 动作排序的主要评分项

```

1 value = SQUARE_TABLE[index] * 8
2 if move & CORNER_MASK:
3     value += 200000
4 if move == priority_move:
5     value += 100000
6 if move & DIAGONAL_DANGER:
7     value -= 6000
8 elif move & EDGE_DANGER:
9     value -= 2500
10
11 next_us, next_them = __play(us, them, move)
12 value -= __count_bits(__legal_moves(next_them, next_us)) * 80
13 value += (__count_bits(next_us) - __count_bits(us)) * 8

```

2.5 动态时间与残局精算

非残局阶段搜索深度上限保持为 $\text{max_depth} = 12$ 。当空位很少时，程序会关闭静态评估并搜索到终局，终局分数使用 $\pm 10^6 + \text{净胜子}$ ，使胜负优先于普通启发式分数。

表 2.2: 时间预算与残局阈值

剩余空位	时间预算	通常深度	说明
> 50	1.2 s	12	开局分支多，快速选择稳健步
41–50	2.2 s	12	开始增加搜索时间
29–40	3.5 s	12	中盘需要更细判断
17–28	4.2 s	12	兼顾深度和响应时间
11–16	4.8 s	12	接近残局，准备精算
≤ 10	8.0 s	空位数	直接搜索到终局

另外，当空位不超过 14 且合法步数不超过 6 时，残局阈值提高到 14，用于捕捉分支较窄的可精确求解局面。

3 代码内容

项目提交文件为 `main.py`，核心结构如下：

- 常量区：棋盘掩码、角点、危险位、边线、位置权重表和八方向移动表；
- 位棋盘工具函数：`_shift`、`_legal_moves`、`_play`、`_board_to_bitboards`；
- `AIPlayer`：保存颜色、搜索深度、置换表、时间控制和当前最优步；
- 搜索函数：`_root_search` 与 `_pvs`；
- 评估与排序函数：`_evaluate`、`_rank_moves`、`_corner_risk`、`_frontier_balance`。

完整源码见附录。正文中只保留关键片段，以便突出实现思路。

4 实验结果

使用平台提供的初级、中级和高级机器人进行先手、后手共六组测试。根据重新测试后保存的截图结果，当前程序六局均获胜，且在多个对局中保持了明显的终局子数优势。

表 4.1: 对战结果汇总

对手难度与执棋	用时	结果	领先棋子数
初级机器人，执黑	71 s	黑棋获胜	60
初级机器人，执白	66 s	白棋获胜	50
中级机器人，执黑	100 s	黑棋获胜	34
中级机器人，执白	85 s	白棋获胜	32
高级机器人，执黑	92 s	黑棋获胜	44
高级机器人，执白	80 s	白棋获胜	59



图 4.1: 初级机器人对战结果



图 4.2: 中级机器人对战结果

测试详情 展示棋盘				测试详情 展示棋盘			
测试点	状态	时长	结果	测试点	状态	时长	结果
对手对弈	✓	92s	黑棋获胜, 领先棋子数: 44	对手对弈	✓	80s	白棋获胜, 领先棋子数: 59

(a) 高级, 执黑

(b) 高级, 执白

图 4.3: 高级机器人对战结果

从结果看，程序不仅能够击败不同难度的机器人，而且先手和后手均有稳定表现。中级机器人两局的领先子数相对更接近，说明其中盘对抗对搜索排序和局面评估提出了更高要求；高级机器人两局则显示程序在残局阶段仍能保持较强的收束能力，尤其执白测试中取得了较大的最终优势。整体而言，角点控制、行动力压制和残局精算仍是本版本获胜的主要来源。

5 总结

本项目最终实现了一个面向黑白棋的确定性搜索智能体。相比最直接的随机 MCTS，当前版本将主要计算量用于更可控的 PVS 博弈树搜索，并用位棋盘降低了合法步生成与局面更新的常数开销。置换表、动作排序和动态时间预算共同保证了搜索深度；分阶段评估函数则把黑白棋中重要的棋理特征显式引入程序。

从实验结果看，当前程序达到了预期目标，能够在初级、中级、高级机器人测试中全部获胜。实现中最关键的经验是：黑白棋不能只追求当前翻子数，前中期更重要的是角点、行动力和避免送角；到残局阶段，静态评估的可信度下降，应尽可能切换到精确搜索。

仍可继续改进的方向包括：

- 使用 Zobrist 哈希压缩置换表键，减少 Python 元组键的哈希开销；
- 加入更准确的稳定子识别，而不是只用边线数量近似稳定性；
- 建立开局库，减少前几步搜索的不确定性；
- 通过自对弈或网格搜索自动调整评估函数权重，减少人工调参偏差。

总体而言，本次实验使我更清楚地理解了博弈程序中算法思想和工程效率之间的关系：同样是向前搜索，只有当棋盘表示、走法排序、剪枝策略和时间控制共同配合时，算法才能在平台限制下稳定转化为实际棋力。

A 完整提交源码

本附录收录当前项目的 `main.py`，即平台提交时使用的完整智能体代码。正文中的算法说明、伪代码、参数表和实验分析均以该文件为准。

Code Listing 5: `main.py` 完整代码

```

1  import time
2
3
4  FULL_BOARD = (1 << 64) - 1
5  KEEP_FROM_A = 0xFEFEFEFEFEFEFEFEFE
6  KEEP_FROM_H = 0x7F7F7F7F7F7F7F7F
7  WIN_SCORE = 10 ** 6
8  SEARCH_INF = 10 ** 9
9  BYTE_COUNTS = tuple(bin(i).count("1") for i in range(256))
10
11  CORNER_MASK = (1 << 0) | (1 << 7) | (1 << 56) | (1 << 63)
12  DIAGONAL_DANGER = (1 << 9) | (1 << 14) | (1 << 49) | (1 << 54)
13  EDGE_DANGER = (
14      (1 << 1) | (1 << 8) | (1 << 6) | (1 << 15) |
15      (1 << 48) | (1 << 57) | (1 << 55) | (1 << 62)
16  )
17  EDGE_MASK = 0xFF818181818181FF
18  OPEN_CORNER_AREAS = (
19      (1 << 0, (1 << 1) | (1 << 8) | (1 << 9)),
20      (1 << 7, (1 << 6) | (1 << 14) | (1 << 15)),
21      (1 << 56, (1 << 48) | (1 << 49) | (1 << 57)),
22      (1 << 63, (1 << 54) | (1 << 55) | (1 << 62)),
23  )
24
25  SQUARE_TABLE = (
26      120, -20, 20, 5, 5, 20, -20, 120,
27      -20, -50, -2, -2, -2, -2, -50, -20,
28      20, -2, 15, 3, 3, 15, -2, 20,
29      5, -2, 3, 3, 3, 3, -2, 5,
30      5, -2, 3, 3, 3, 3, -2, 5,
31      20, -2, 15, 3, 3, 15, -2, 20,
32      -20, -50, -2, -2, -2, -2, -50, -20,
33      120, -20, 20, 5, 5, 20, -20, 120,
34  )
35

```

```
36 DIRECTIONS = (  
37     (1, KEEP_FROM_A),  
38     (-1, KEEP_FROM_H),  
39     (8, FULL_BOARD),  
40     (-8, FULL_BOARD),  
41     (9, KEEP_FROM_A),  
42     (7, KEEP_FROM_H),  
43     (-7, KEEP_FROM_A),  
44     (-9, KEEP_FROM_H),  
45 )  
46  
47  
48 class SearchTimeout(Exception):  
49     pass  
50  
51  
52 def __shift(bits, step, file_mask):  
53     if step > 0:  
54         return ((bits << step) & file_mask) & FULL_BOARD  
55     return (bits >> -step) & file_mask  
56  
57  
58 def __each_bit(mask):  
59     while mask:  
60         low_bit = mask & -mask  
61         yield low_bit  
62         mask ^= low_bit  
63  
64  
65 def __legal_moves(us, them):  
66     blank = FULL_BOARD ^ (us | them)  
67     choices = 0  
68     for step, file_mask in DIRECTIONS:  
69         captured_line = __shift(us, step, file_mask) & them  
70         for _ in range(5):  
71             captured_line |= __shift(captured_line, step, file_mask) & them  
72             choices |= __shift(captured_line, step, file_mask) & blank  
73     return choices & FULL_BOARD  
74  
75  
76 def __play(us, them, move):
```

```

77     flipped = 0
78     for step, file_mask in DIRECTIONS:
79         ray = 0
80         cursor = __shift(move, step, file_mask)
81         while cursor & them:
82             ray |= cursor
83             cursor = __shift(cursor, step, file_mask)
84         if cursor & us:
85             flipped |= ray
86     return (us | move | flipped) & FULL_BOARD, them & ~flipped
87
88
89 def __board_to_bitboards(board):
90     dark = 0
91     light = 0
92     for r in range(8):
93         row_offset = r << 3
94         for c, stone in enumerate(board.__board[r]):
95             place = 1 << (row_offset + c)
96             if stone == "X":
97                 dark |= place
98             elif stone == "O":
99                 light |= place
100     return dark, light
101
102
103 def __move_name(move):
104     index = move.bit_length() - 1
105     return chr(ord("A") + (index & 7)) + str((index >> 3) + 1)
106
107
108 def __neighbor_cells(mask):
109     around = 0
110     for step, file_mask in DIRECTIONS:
111         around |= __shift(mask, step, file_mask)
112     return around & FULL_BOARD
113
114
115 def __ratio_diff(left, right):
116     total = left + right
117     return 0 if total == 0 else 100 * (left - right) // total

```

```

118
119
120 def __count_bits(value):
121     return (
122         BYTE_COUNTS[value & 0xFF] +
123         BYTE_COUNTS[(value >> 8) & 0xFF] +
124         BYTE_COUNTS[(value >> 16) & 0xFF] +
125         BYTE_COUNTS[(value >> 24) & 0xFF] +
126         BYTE_COUNTS[(value >> 32) & 0xFF] +
127         BYTE_COUNTS[(value >> 40) & 0xFF] +
128         BYTE_COUNTS[(value >> 48) & 0xFF] +
129         BYTE_COUNTS[(value >> 56) & 0xFF]
130     )
131
132
133 def __weight_sum(us, them):
134     value = 0
135     for stones, sign in ((us, 1), (them, -1)):
136         rest = stones
137         while rest:
138             bit = rest & -rest
139             value += sign * SQUARE_TABLE[bit.bit_length() - 1]
140             rest ^= bit
141     return value
142
143
144 class AIPlayer:
145     def __init__(self, color):
146         self.color = color
147         self.max_depth = 12
148         self.transposition = {}
149         self.stop_at = 0.0
150         self.visited_nodes = 0
151         self.best_move = 0
152
153     def __move_time_limit(self, empty_count):
154         if empty_count > 50:
155             return 1.2
156         if empty_count > 40:
157             return 2.2
158         if empty_count > 28:

```

```

159         return 3.5
160     if empty_count > 16:
161         return 4.2
162     if empty_count > 10:
163         return 4.8
164     return 8.0
165
166     def __exact_limit(self, empty_count, branch_count):
167         if empty_count <= 14 and branch_count <= 6:
168             return 14
169         return 12
170
171     def __iterative_limit(self, empty_count, exact_limit):
172         if empty_count <= exact_limit:
173             return empty_count
174         return self.max_depth
175
176     def get_move(self, board):
177         side_name = "黑棋" if self.color == "X" else "白棋"
178         print("请等一会, 对方 {}-{} 正在思考中...".format(side_name, self.color))
179
180         black, white = __board_to_bitboards(board)
181         us = black if self.color == "X" else white
182         them = white if self.color == "X" else black
183         legal = __legal_moves(us, them)
184         if legal == 0:
185             return None
186         if legal & (legal - 1) == 0:
187             return __move_name(legal)
188
189         forced_corners = legal & CORNER_MASK
190         if forced_corners:
191             corner_order = self.__rank_moves(us, them, forced_corners, 0)
192             self.best_move = corner_order[0]
193             return __move_name(corner_order[0])
194
195         empty_count = 64 - __count_bits(us | them)
196         branch_count = __count_bits(legal)
197         exact_limit = self.__exact_limit(empty_count, branch_count)
198         self.stop_at = time.perf_counter() + min(self.__move_time_limit(
            empty_count), 55.0)

```



```

199     self.visited_nodes = 0
200     self.transposition.clear()
201
202     ordered = self.__rank_moves(us, them, legal, 0)
203     chosen = ordered[0]
204     finished_depth = 0
205
206     try:
207         max_depth = min(self.__iterative_limit(empty_count, exact_limit),
208                         empty_count)
209         for depth in range(1, max_depth + 1):
210             candidate, __ = self.__root_search(us, them, depth, chosen)
211             if candidate:
212                 chosen = candidate
213                 finished_depth = depth
214                 if empty_count <= exact_limit and finished_depth >= empty_count:
215                     break
216         except SearchTimeout:
217             pass
218
219     self.best_move = chosen
220     return __move_name(chosen)
221
222 def __guard_clock(self):
223     self.visited_nodes += 1
224     if self.visited_nodes & 1023 == 0 and time.perf_counter() >= self.stop_at:
225         raise SearchTimeout
226
227 def __final_score(self, us, them):
228     margin = __count_bits(us) - __count_bits(them)
229     if margin > 0:
230         return WIN_SCORE + margin
231     if margin < 0:
232         return -WIN_SCORE + margin
233     return 0
234
235 def __corner_risk(self, us, them, occupied):
236     risk = 0
237     for corner, near_corner in OPEN_CORNER_AREAS:
238         if occupied & corner:

```

```

239         risk += __count_bits(them & near_corner) - __count_bits(us &
240             near_corner)
241     return risk
242
243 def __frontier_balance(self, us, them, occupied):
244     empty = FULL_BOARD ^ occupied
245     frontier = __neighbor_cells(empty)
246     our_frontier = __count_bits(us & frontier)
247     their_frontier = __count_bits(them & frontier)
248     return __ratio_diff(their_frontier, our_frontier)
249
250 def __evaluate(self, us, them, legal):
251     occupied = us | them
252     empty_count = 64 - __count_bits(occupied)
253     reply = __legal_moves(them, us)
254
255     mobility = __ratio_diff(__count_bits(legal), __count_bits(reply))
256     disc_delta = __ratio_diff(__count_bits(us), __count_bits(them))
257     square_score = __weight_sum(us, them)
258     corners = 25 * (__count_bits(us & CORNER_MASK) - __count_bits(them
259         & CORNER_MASK))
260     edge_score = __count_bits(us & EDGE_MASK) - __count_bits(them &
261         EDGE_MASK)
262     frontier = self.__frontier_balance(us, them, occupied)
263     corner_risk = self.__corner_risk(us, them, occupied)
264     parity = 1 if empty_count & 1 else -1
265
266     if empty_count > 40:
267         return (
268             10 * mobility +
269             20 * corners +
270             12 * corner_risk +
271             4 * square_score +
272             3 * frontier +
273             edge_score
274         )
275
276     if empty_count > 16:
277         return (
278             8 * mobility +
279             30 * corners +

```

```

277         10 * corner_risk +
278         3 * frontier +
279         3 * square_score +
280         2 * edge_score +
281         2 * disc_delta
282     )
283
284     return (
285         5 * mobility +
286         35 * corners +
287         4 * corner_risk +
288         4 * frontier +
289         2 * square_score +
290         4 * edge_score +
291         12 * disc_delta +
292         8 * parity
293     )
294
295     def __rank_moves(self, us, them, legal, priority_move):
296         scored = []
297         for move in __each_bit(legal):
298             index = move.bit_length() - 1
299             value = SQUARE_TABLE[index] * 8
300             if move & CORNER_MASK:
301                 value += 200000
302             if move == priority_move:
303                 value += 100000
304             if move & DIAGONAL_DANGER:
305                 value -= 6000
306             elif move & EDGE_DANGER:
307                 value -= 2500
308
309             next_us, next_them = __play(us, them, move)
310             value -= __count_bits(__legal_moves(next_them, next_us)) * 80
311             value += (__count_bits(next_us) - __count_bits(us)) * 8
312             scored.append((value, move))
313
314         scored.sort(reverse=True)
315         return [move for __, move in scored]
316
317     def __root_search(self, us, them, depth, priority_move):

```

```

318     legal = __legal_moves(us, them)
319     ordered = self.__rank_moves(us, them, legal, priority_move)
320     alpha = -SEARCH_INF
321     beta = SEARCH_INF
322     best_value = -SEARCH_INF
323     best_move = ordered[0]
324
325     for i, move in enumerate(ordered):
326         self.__guard_clock()
327         next_us, next_them = __play(us, them, move)
328         if i == 0:
329             value = -self.__pvs(next_them, next_us, depth - 1, -beta, -alpha,
330                                 False)
331         else:
332             value = -self.__pvs(next_them, next_us, depth - 1, -alpha - 1, -
333                                 alpha, False)
334             if alpha < value < beta:
335                 value = -self.__pvs(next_them, next_us, depth - 1, -beta, -
336                                     value, False)
337             if value > best_value:
338                 best_value = value
339                 best_move = move
340             if value > alpha:
341                 alpha = value
342
343     return best_move, best_value
344
345 def __pvs(self, us, them, depth, alpha, beta, passed):
346     self.__guard_clock()
347     original_alpha = alpha
348     key = (us, them, passed)
349     cached = self.transposition.get(key)
350     cached_move = 0
351
352     if cached is not None:
353         saved_depth, bound_type, saved_value, cached_move = cached
354         if saved_depth >= depth:
355             if bound_type == 0:
356                 return saved_value
357             if bound_type == 1 and saved_value > alpha:
358                 alpha = saved_value

```

```

356         elif bound_type == 2 and saved_value < beta:
357             beta = saved_value
358         if alpha >= beta:
359             return saved_value
360
361     legal = __legal_moves(us, them)
362     if legal == 0:
363         if passed:
364             return self.__final_score(us, them)
365         value = -self.__pvs(them, us, depth, -beta, -alpha, True)
366         self.transposition[key] = (depth, 0, value, 0)
367         return value
368
369     occupied = us | them
370     empty_count = 64 - __count_bits(occupied)
371     exact_limit = self.__exact_limit(empty_count, __count_bits(legal))
372     if depth <= 0 and empty_count > exact_limit:
373         return self.__evaluate(us, them, legal)
374
375     best_value = -SEARCH_INF
376     best_move = 0
377     ordered = self.__rank_moves(us, them, legal, cached_move)
378
379     for i, move in enumerate(ordered):
380         next_us, next_them = __play(us, them, move)
381         if i == 0:
382             value = -self.__pvs(next_them, next_us, depth - 1, -beta, -alpha,
383                                 False)
384         else:
385             value = -self.__pvs(next_them, next_us, depth - 1, -alpha - 1, -
386                                 alpha, False)
387             if alpha < value < beta:
388                 value = -self.__pvs(next_them, next_us, depth - 1, -beta, -
389                                     value, False)
390             if value > best_value:
391                 best_value = value
392                 best_move = move
393             if value > alpha:
394                 alpha = value
395             if alpha >= beta:
396                 break

```

```
394
395     bound_type = 0
396     if best_value <= original_alpha:
397         bound_type = 2
398     elif best_value >= beta:
399         bound_type = 1
400     self.transposition[key] = (depth, bound_type, best_value, best_move)
401     return best_value
```